Technická fakulta

# Systém pro správu skladu elektronických součástek

| | |
|---|---|
| **Předmět:** | Projektování IS |
| **Kód předmětu:** | EIT21E |
| **Autoři:** | Tomáš Simandl |
| | Tomáš Szetei |
| | Lukáš Kvapil |
| **Datum:** | 28. 4. 2026 |

# Obsah


**1 Formulace problému** **3**

1.1 Účel . . . 3

1.2 Specifikace . . . 3

**2 Datový slovník** **4**

2.1 Soucastka . . . 4

2.2 Projekt . . . 4

2.3 SkladovyPohyb . . . 5

2.4 PrijemPohyb . . . 5

2.5 VydejPohyb . . . 5

2.6 PresunutiPohyb . . . 6

2.7 Sklad . . . 6

2.8 Lokace . . . 6

2.9 Uzivatel . . . 6

**3 Objektový model** **7**

**4 Dynamický model** **9**

4.1 Stavový diagram třídy SkladovyPohyb . . . 9

4.2 Stavový diagram třídy Soucastka . . . 11

**5 Model interakcí** **12**

5.1 Use Case diagram . . . 12

5.2 Scénáře . . . 13

5.2.1 Scénář: Příjem na sklad (PrijemPohyb) . . . 13

5.2.2 Scénář: Výdej ze skladu (VydejPohyb) . . . 13

5.2.3 Scénář: Přesun součástky (PresunutiPohyb) . . . 13

5.2.4 Scénář: Přiřazení součástky k projektu . . . 14

5.2.5 Scénář: Vyhledání součástky ve skladu . . . 14

5.3 Sekvenční diagram – Vytvoření pohybu . . . 14

5.4 Diagram aktivit – Vytvoření pohybu . . . 17

**6 Závěr** **19**

**A Zdrojové kódy PlantUML diagramů** **20**
A.1 Use Case diagram . . . . . 20
A.2 Class diagram . . . . . 21
A.3 Stavový diagram – SkladovyPohyb . . . . . 22
A.4 Stavový diagram – Soucastka . . . . . 23
A.5 Sekvenční diagram – Vytvoření pohybu . . . . . 24
A.6 Diagram aktivit – Vytvoření pohybu . . . . . 25

# 1. Formulace problému

## 1.1 Účel

Cílem tohoto projektu je teoreticky navrhnout informační systém pro správu skladových součástek. Systém má umožňovat evidenci součástek, jejich kategorií, dodavatelů a skladových lokací. Dále má podporovat sledování pohybů součástek ve skladu, jako je naskladnění, vyskladnění nebo přesun mezi jednotlivými lokacemi.

Součástí systému je také možnost přiřazovat součástky k projektům a evidovat jejich množství potřebné pro daný projekt.

## 1.2 Specifikace

Návrh systému je znázorněn pomocí UML diagramů, které popisují jednotlivé třídy, jejich atributy, vztahy mezi nimi a chování systému při klíčových operacích. Systém zahrnuje objektový model (class diagram), dynamický model (stavové diagramy) a model interakcí (use case diagram, scénáře, sekvenční a aktivity diagram).

# 2. Datový slovník

## 2.1 Soucastka

Skladová položka evidovaná v systému WMS s aktuálním množstvím na skladě. Atribut rezervovaneMnozstvi sleduje, kolik kusů je aktuálně blokováno pro projekty – součástka tak může být Dostupna i s částečnou rezervací zároveň. Součástka může nabývat stavů: Dostupna (k dispozici, případně s rezervovanou částí), Vydana (fyzicky vydána ze skladu) nebo Odepsana (trvale vyřazena z evidence).

**Atributy:**

- id: int
- nazev: String
- mnozstvi: int
- rezervovaneMnozstvi: int
- stav: String (Dostupna | Vydana | Odepsana)

**Metody:**

- zvysMnozstvi(pocet: int)
- snizMnozstvi(pocet: int)
- rezervovat(pocet: int)
- zrusitRezervaci(pocet: int)
- overDostupnost(): boolean
- odepsat() – nastaví stav na Odepsana a zavolá snizMnozstvi() pro vyřazení z aktivních zásob

## 2.2 Projekt

Projekt, ke kterému jsou přiřazeny součástky pro sledování spotřeby.

**Atributy:**

- id: int
- nazev: String

**Metody:**

- priradSoucastku(Soucastka)
- odeberSoucastku(Soucastka)

## 2.3 SkladovyPohyb

Abstraktní třída reprezentující pohyb součástek – příjem, výdej nebo přesun. Každý pohyb prochází životním cyklem: vytvoření, validace dat a provedení pohybu.

**Atributy:**

- id: int
- datum: Date
- stav: String (Vytvoren | Validovan | Proveden | Zamitnut)
- mnozstvi: int

**Metody:**

- vytvoritPohyb(soucastka, mnozstvi)
- overitData()
- provestPohyb()

## 2.4 PrijemPohyb

Konkrétní pohyb typu příjem – naskladnění součástek od dodavatele. Dědí ze SkladovyPohyb.

**Atributy:**

- dodavatel: String – název dodavatele součástek
- dědí ze SkladovyPohyb: id, datum, stav, mnozstvi

**Metody:**

- vytvoritPohyb(soucastka, mnozstvi)

## 2.5 VydejPohyb

Konkrétní pohyb typu výdej – vyskladnění součástek ze skladu. Dědí ze SkladovyPohyb. Atribut duvod popisuje účel výdeje, např. použití v projektu, reklamace, interní spotřeba nebo oprava.

**Atributy:**

- duvod: String – účel výdeje (projekt, reklamace, interní spotřeba, oprava)
- dědí ze SkladovyPohyb: id, datum, stav, mnozstvi

**Metody:**

- vytvoritPohyb(soucastka, mnozstvi)

## 2.6 PresunutiPohyb

Pohyb typu přesun – přemístnění součástek mezi lokacemi ve skladu. Dědí ze SkladovyPohyb.

**Atributy:**

- zdrojLokaceId: int – ID zdrojové lokace
- cilLokaceId: int – ID cílové lokace
- dědí ze SkladovyPohyb: id, datum, stav, mnozstvi

**Metody:**

- vytvoritPohyb(soucastka, mnozstvi)
- zmenLokaci()

## 2.7 Sklad

Fyzický sklad rozdělený na lokace, ve kterých jsou uloženy součástky.

**Atributy:**

- id: int
- nazev: String

**Metody:** najdiSoucastku(id: int): Soucastka

## 2.8 Lokace

Konkrétní místo v skladu identifikované označením (např. A-01-03).

**Atributy:**

- id: int
- oznaceni: String

## 2.9 Uzivatel

Systém rozlišuje dvě role:

- **Skladnik** – provádí příjem, výdej a přesun součástek a kontroluje stav zásob.
- **VedouciSkladu** – spravuje projekty, přiřazuje součástky k projektům a odebírá je z projektů.

**Atributy:**

- id: int
- jmeno: String
- email: String
- role: String (Skladnik | VedouciSkladu)

# 3. Objektový model

Class diagram ukazuje hlavní třídy systému WMS a jejich vztahy. Součástky jsou fyzicky uloženy v lokacích a mohou být přiřazeny k projektům. Nové pohyby vznikají přes SkladovyPohyb.vytvoritPohyb() a součástky se k projektům přiřazují přes Projekt.priradSoucastku().

**Generalizace (zobecnění):** Abstraktní třída SkladovyPohyb je základem pro tři konkrétní podtřídy – PrijemPohyb (naskladnění), VydejPohyb (vyskladnění) a PresunutiPohyb (přesun mezi lokacemi), které dědí společné atributy a přidávají vlastní specifické atributy.

**Asociace:** Třídy jsou propojeny asociacemi vyjadřujícími vztahy mezi objekty – například SkladovyPohyb je vázán na Soucastka (pohyb se týká konkrétní součástky) nebo Skladnik vytváří SkladovyPohyb a VedouciSkladu spravuje Projekt.

**Agregace:** Sklad sdružuje Lokace – lokace jsou součástí skladu, ale mohou existovat i samostatně.

**Kardinalita:** Každá vazba je označena multiplicitou: pohyb se vždy týká právě jedné součástky ($* \rightarrow 1$), jeden skladník může vytvořit libovolný počet pohybů ($1 \rightarrow *$).

**Kvalifikovaná vazba:** Vazba mezi Sklad a Soucastka je kvalifikovaná – sklad přistupuje ke konkrétní součástce prostřednictvím kvalifikátoru id: int.

**Vazební třída a atribut spojení:** Vztah mezi Projekt a Soucastka je realizován pomocí vazební třídy, která nese atribut spojení pocet: int – tedy kolik kusů dané součástky je přiřazeno k projektu.

Třída SkladovyPohyb deklaruje abstraktní metodu vytvoritPohyb(), kterou každá podtřída implementuje po svém:

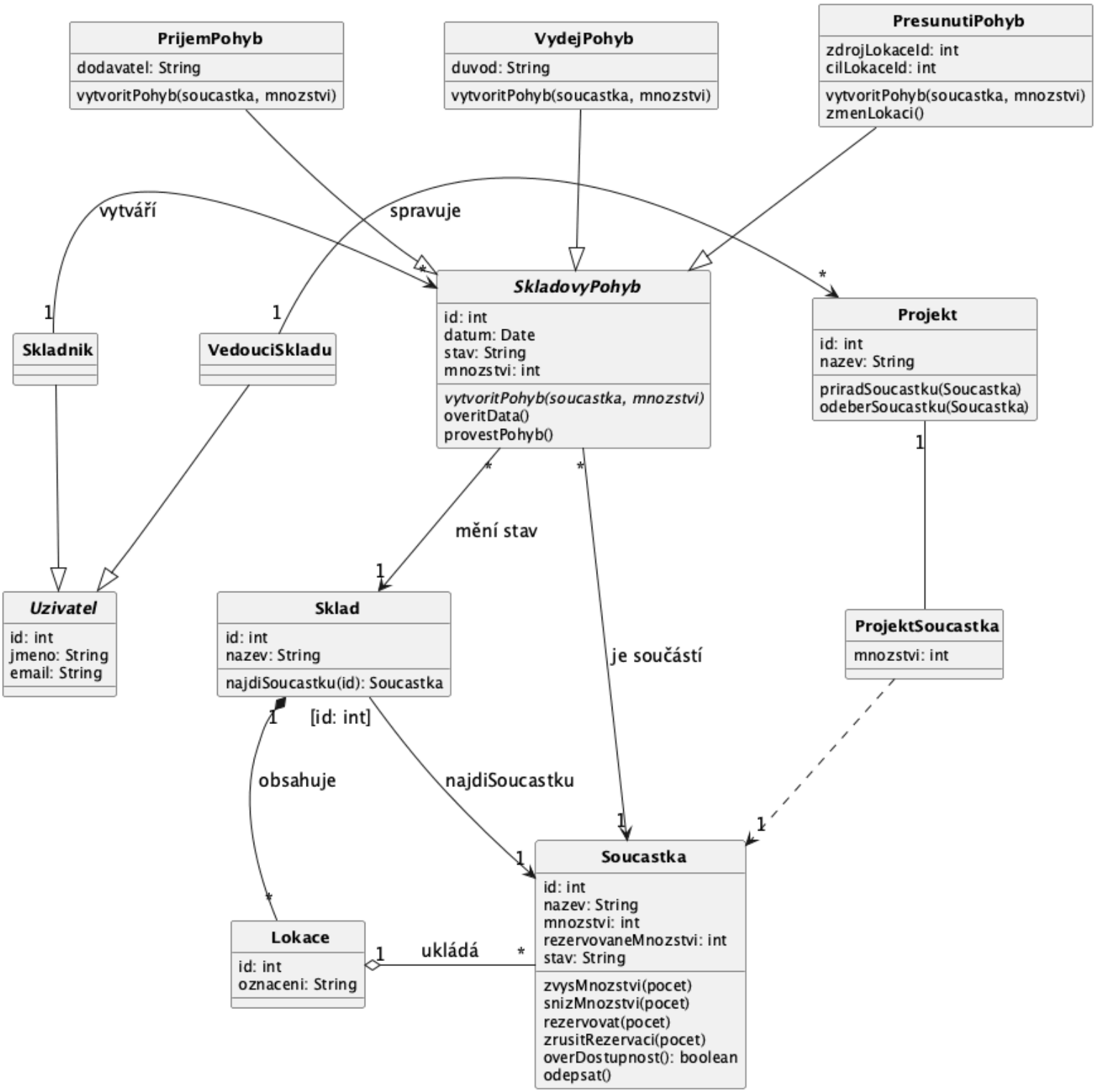
- PrijemPohyb.vytvoritPohyb() – vytvoří příjem a nastaví atribut dodavatel.
- VydejPohyb.vytvoritPohyb() – vytvoří výdej a nastaví atribut duvod.
- PresunutiPohyb.vytvoritPohyb() – vytvoří přesun a nastaví zdrojLokaceId a cilLokaceId.

Třída SkladovyPohyb zaznamenává každou změnu stavu zásob v systému. Konkrétně:

- PrijemPohyb – příjem součástek na sklad od dodavatele; zvyšuje množství dané součástky.
- VydejPohyb – výdej součástek ze skladu na projekt; snižuje množství dané součástky.
- PresunutiPohyb – přemístnění součástek mezi lokacemi ve skladu; mění přiřazenou lokaci součástky.

Každý pohyb se váže na konkrétní součástku (vazba je součástí, multiplicita $* \rightarrow 1$) a mění stav jednoho skladu (vazba mění stav, multiplicita $* \rightarrow 1$). Jeden skladník může vytvořit libovolný

počet pohybů (vazba vytváří, multiplicita $1 \rightarrow *$). Pohyb nemění fyzickou lokaci součástky – ta je pevně dána vazbou Lokace → Soucastka.



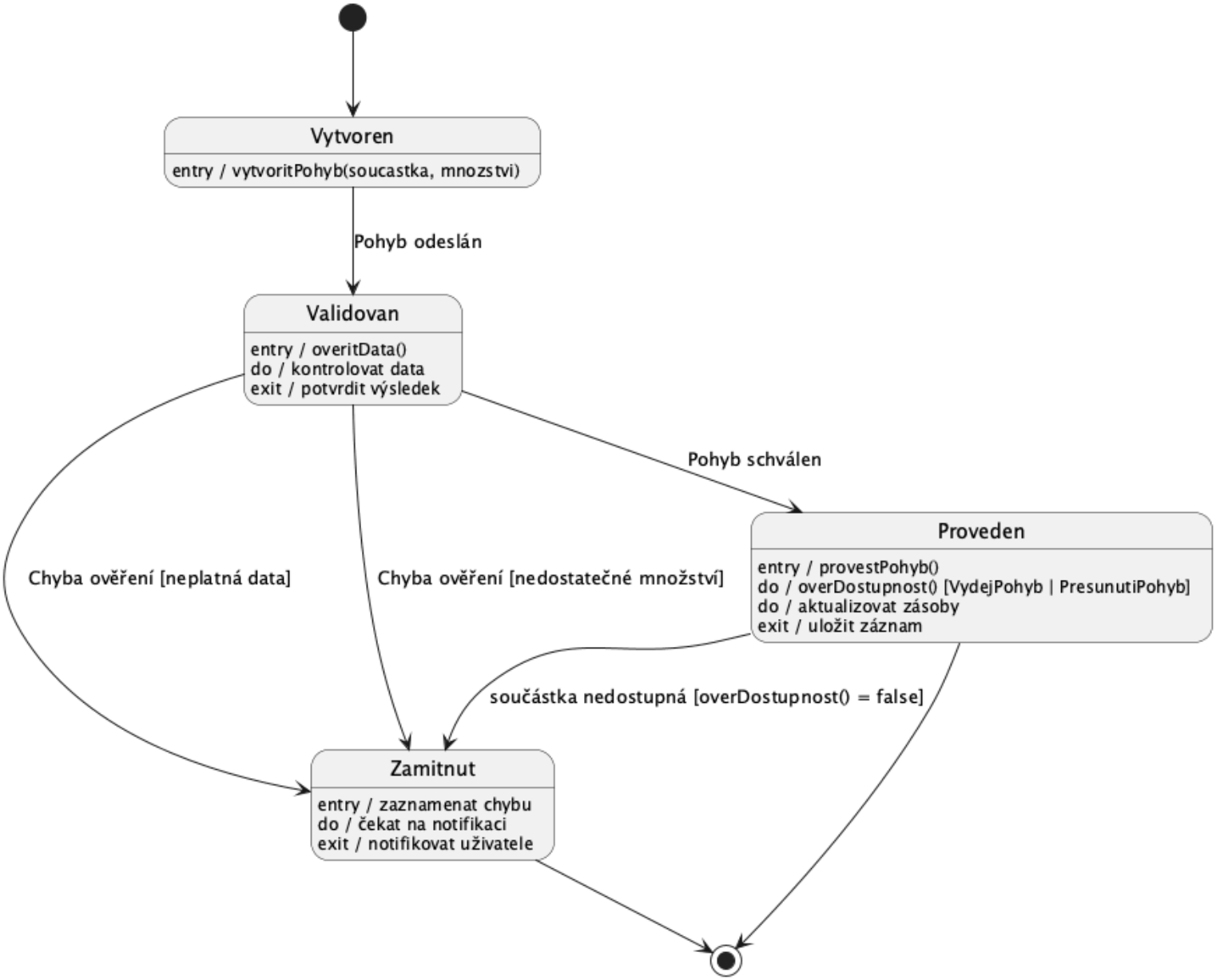

Obrázek 3.1: Class diagram

# 4. Dynamický model

## 4.1 Stavový diagram třídy SkladovyPohyb

Třída SkladovyPohyb reprezentuje obecný skladový pohyb – tedy jakoukoli změnu stavu zásob součástek v systému. Každý pohyb prochází životním cyklem řízeným třemi metodami:

- vytvoritPohyb(soucastka, mnozstvi) – inicializuje nový pohyb: přiřadí ID, nastaví datum vzniku, typ pohybu a vazbu na součástku;
- overitData() – ověří platnost zadaných dat: existenci součástky, vyplnění povinných polí, dostatečné množství na skladě (příjem/výdej) nebo platnost lokací (přesun);
- provestPohyb() – fyzicky provede pohyb podle typu:
  - příjem → soucastka.zvysMnozstvi(pocet)
  - výdej → soucastka.snizMnozstvi(pocet)
  - přesun → zmenLokaci()

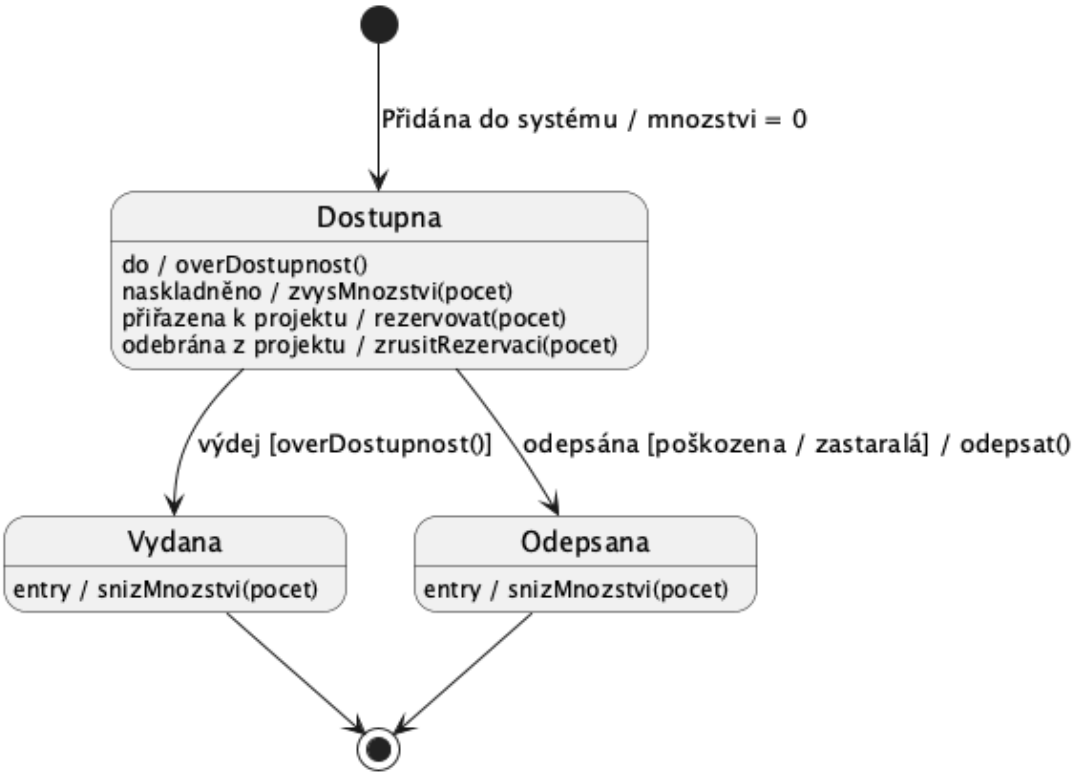



Obrázek 4.1: Stavový diagram třídy SkladovyPohyb

## 4.2 Stavový diagram třídy Soucastka

Diagram zachycuje, jak se mění stav součástky v průběhu skladových operací. Součástka je přidána do systému a okamžitě vstupuje do stavu Dostupna s nulovým množstvím. Stav Dostupna je hlavní pracovní stav součástky – může mít mnozstvi = 0 (čeká na naskladnění) i nenulové množství, přičemž část může být blokována atributem rezervovaneMnozstvi. Metoda overDostupnost() ověří, zda je volné množství (mnozstvi - rezervovaneMnozstvi > 0). Stav součástky je řízen těmito metodami:

- zvysMnozstvi(pocet) – zavolá se při naskladnění; zvýší atribut mnozstvi, součástka zůstává ve stavu Dostupna.
- rezervovat(pocet) – zablokuje část kusů pro projekt; zvýší rezervovaneMnozstvi, stav se nemění.
- zrusitRezervaci(pocet) – uvolní rezervaci; sníží rezervovaneMnozstvi.
- snizMnozstvi(pocet) – zavolá se při výdeji ze skladu (přechod do stavu Vydana).
- odepsat() – zavolá se při vyřazení součástky (přechod do stavu Odepsana); nastaví stav na Odepsana a interně spustí snizMnozstvi() pro odečtení množství ze skladu.

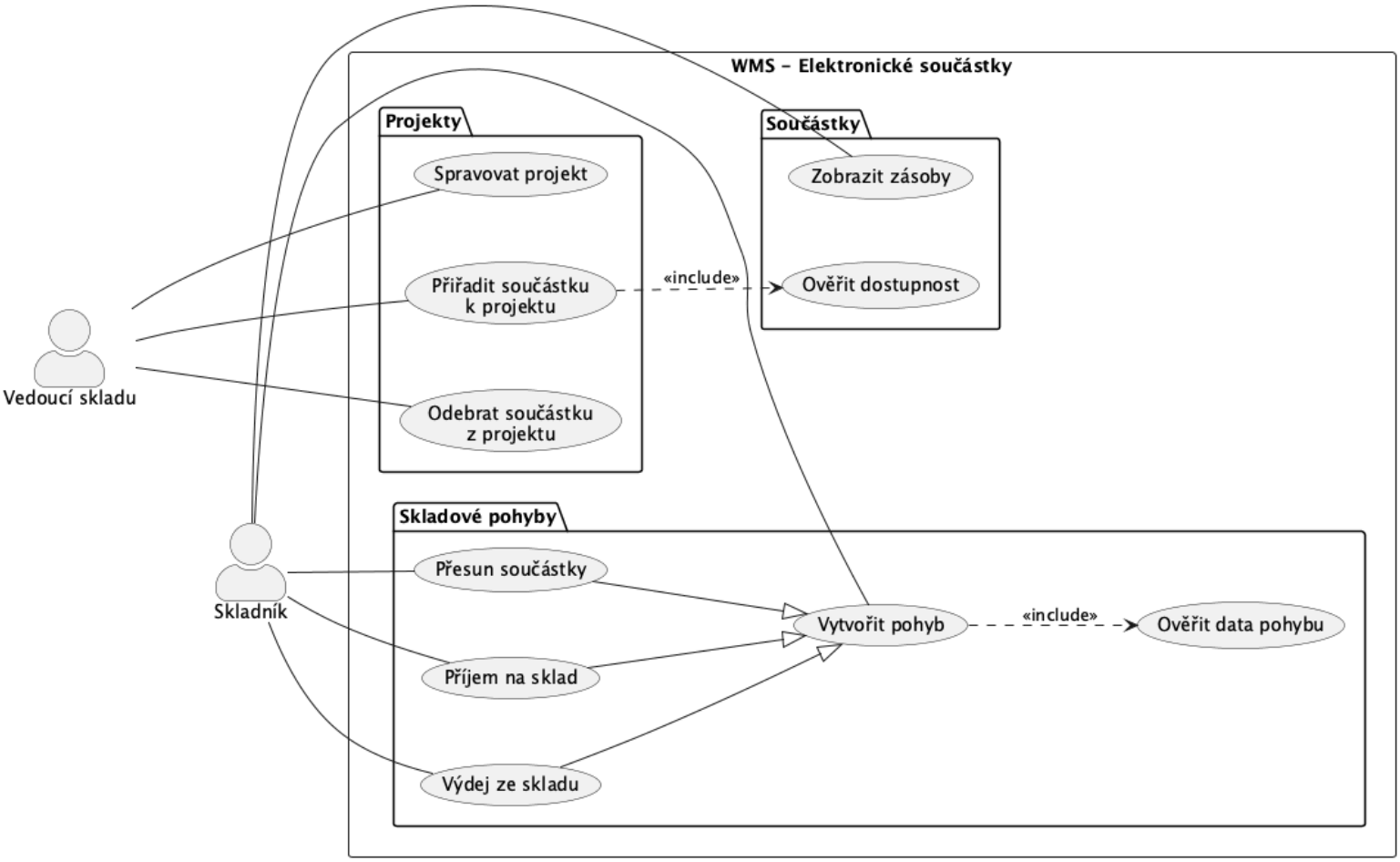



Obrázek 4.2: Stavový diagram třídy Soucastka

# 5. Model interakcí

## 5.1 Use Case diagram

**Vedoucí skladu** spravuje projekty – přiřazuje součástky k projektům a odebírá je. **Skladník** provádí příjem, výdej a přesun součástek a kontroluje stav zásob.




Obrázek 5.1: Use Case diagram

## 5.2 Scénáře

### 5.2.1 Scénář: Příjem na sklad (PrijemPohyb)

1. Skladník zvolí typ pohybu příjem a zadá součástku, množství a dodavatele.
2. Systém volá overitData() – ověří existenci součástky a platnost cílové lokace.
3. Pokud data OK → systém volá provestPohyb(), který interně volá Soucastka.zvysMnozstvi().
4. Pohyb je potvrzen a zásoby jsou aktualizovány.
5. **Alternativní tok:** součástka neexistuje nebo lokace je neplatná → systém vrátí chybovou hlášku a pohyb je zamítnut.

### 5.2.2 Scénář: Výdej ze skladu (VydejPohyb)

1. Skladník zvolí typ pohybu výdej a zadá součástku, množství a důvod výdeje.
2. Systém volá overitData() – ověří existenci součástky a zda celkové mnozstvi $\geq$ požadovanému množství.
3. Systém volá Soucastka.overDostupnost() – ověří, zda je dostatek volných (nerezerovaných) kusů.
4. Pokud data OK → systém volá provestPohyb(), který interně volá Soucastka.snizMnozstvi().
5. Pohyb je potvrzen a zásoby jsou sníženy.
6. **Alternativní tok:** nedostatečné množství nebo součástka je rezervována → systém vrátí chybovou hlášku a pohyb je zamítnut.

### 5.2.3 Scénář: Přesun součástky (PresunutiPohyb)

1. Skladník zvolí typ pohybu přesun a zadá součástku, množství, zdrojovou a cílovou lokaci.
2. Systém volá overitData() – ověří existenci součástky, dostatečné množství ve zdrojové lokaci a platnost cílové lokace.
3. Pokud data OK → systém volá provestPohyb(), který interně ověří dostupnost voláním Soucastka.overDostupnost() a poté zavolá PresunutiPohyb.zmenLokaci().
4. Pohyb je potvrzen; součástka je evidována v cílové lokaci.
5. **Alternativní tok:** zdrojová nebo cílová lokace je neplatná, nebo součástka není dostupná → systém vrátí chybovou hlášku a pohyb je zamítnut.

### 5.2.4 Scénář: Přiřazení součástky k projektu

1. Vedoucí skladu vybere projekt a zadá součástku s množstvím.
2. Systém volá Soucastka.overDostupnost() – ověří, zda mnozstvi − rezervovaneMnozstvi $> 0$.
3. Pokud je dostatek volných kusů → systém volá Projekt.priradSoucastku(), které interně zavolá Soucastka.rezervovat().
4. Součástka je přiřazena k projektu a rezervované množství je aktualizováno.
5. **Alternativní tok:** nedostatečný počet volných kusů → systém vrátí chybovou hlášku a přiřazení je zamítnuto.

### 5.2.5 Scénář: Vyhledání součástky ve skladu

1. Skladník zadá ID hledané součástky.
2. Systém volá Sklad.najdiSoucastku(id) – vyhledá součástku podle zadaného ID.
3. Pokud součástka existuje → systém zobrazí její detail: název, aktuální množství, rezervované množství a lokaci.
4. **Alternativní tok:** součástka s daným ID neexistuje → systém vrátí hlášku, že součástka nebyla nalezena.

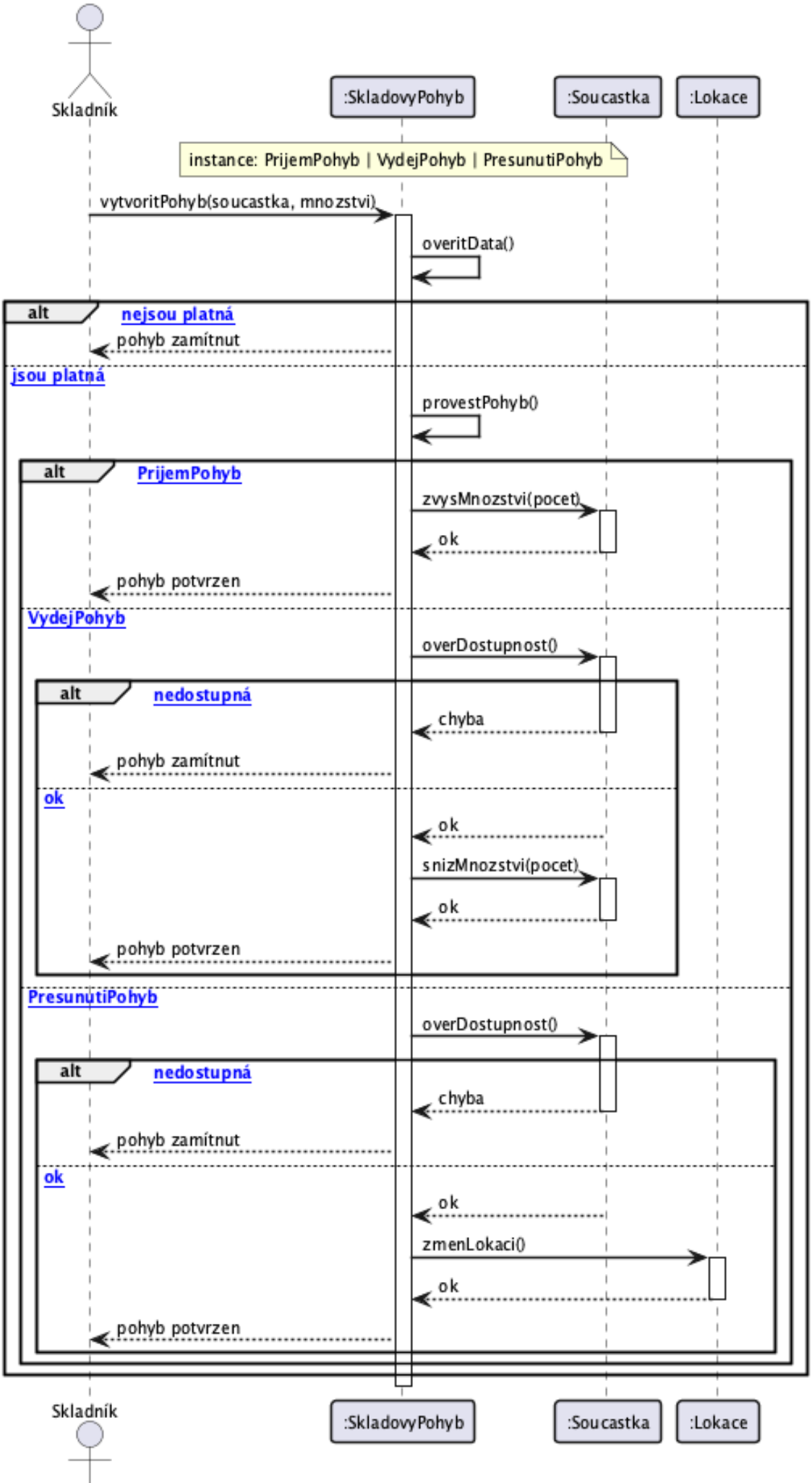
## 5.3 Sekvenční diagram – Vytvoření pohybu

Skladník zadá ID součástky a množství – systém vyhledá součástku voláním Sklad.najdiSoucastku(id) a spustí vytvoření pohybu voláním vytvoritPohyb(soucastka, mnozstvi) na instanci konkrétní podtřídy (PrijemPohyb, VydejPohyb nebo PresunutiPohyb).

Pohyb ověří data voláním overitData() – pokud nejsou platná, je zamítnut. Jinak se zavolá provestPohyb(), které polymorfně provede:

- PrijemPohyb – zavolá Soucastka.zvysMnozstvi(pocet) a zvýší množství na skladě.
- VydejPohyb – nejprve ověří dostupnost voláním Soucastka.overDostupnost(); pokud součástka není dostupná (množství je rezervováno), pohyb je zamítnut. Jinak zavolá Soucastka.snizMnozstvi(pocet) a sníží množství na skladě.
- PresunutiPohyb – ověří dostupnost voláním Soucastka.overDostupnost(); pokud součástka není dostupná, pohyb je zamítnut. Jinak přemístí součástku voláním Lokace.zmenLokaci().

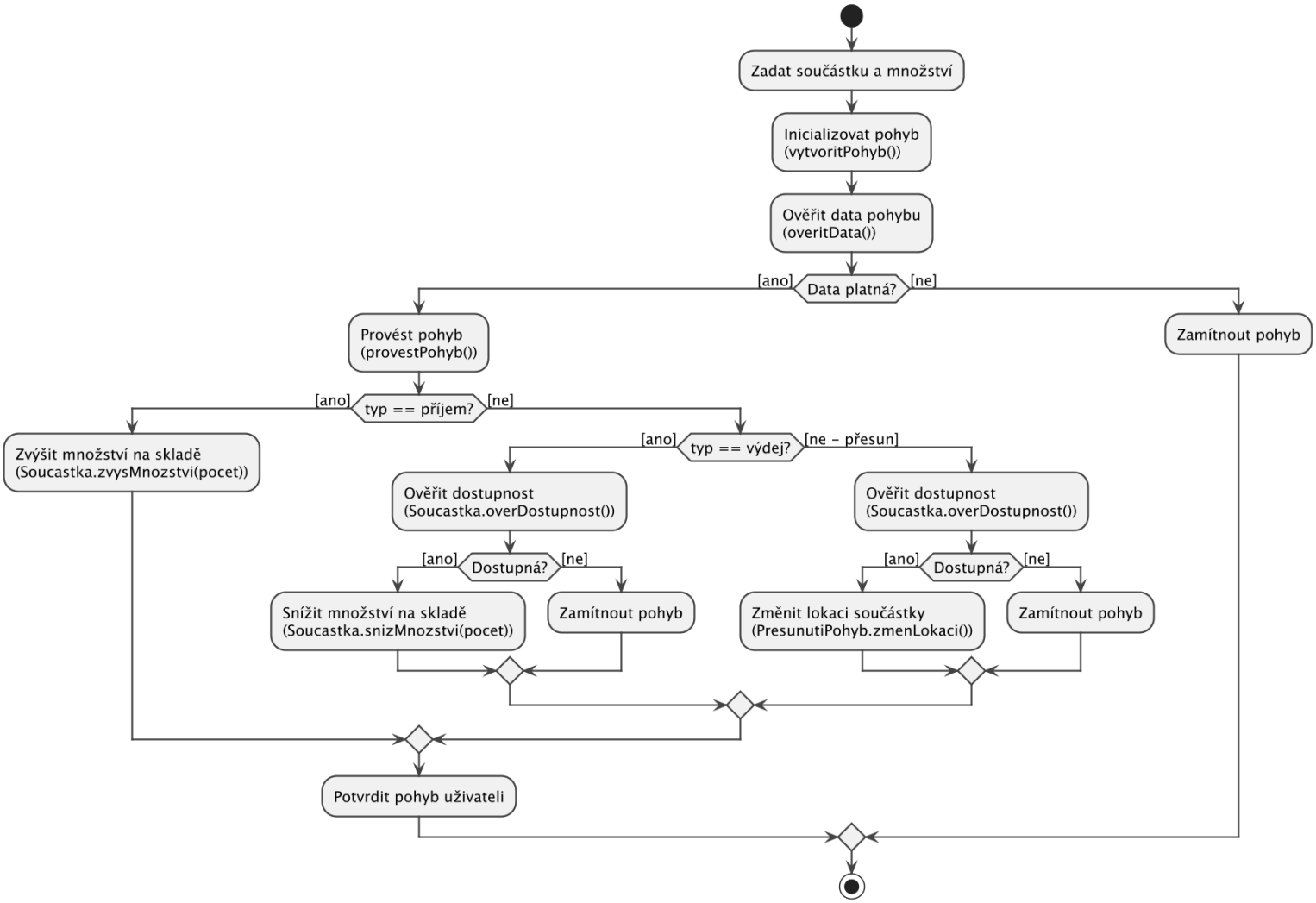
Po úspěšném dokončení systém vrátí Skladníkovi potvrzení pohybu.


Skladník
:SkladovyPohyb
:Soucastka
:Lokace
instance: PrijemPohyb | VydejPohyb | PresunutiPohyb
vytvoritPohyb(soucastka, mnozstvi)
overitData()
alt
nejsou platná
pohyb zamítnut
jsou platná
provestPohyb()
alt
PrijemPohyb
zvysMnozstvi(pocet)
ok
pohyb potvrzen
VydejPohyb
overDostupnost()
alt
nedostupná
chyba
pohyb zamítnut
ok
ok
snizMnozstvi(pocet)
ok
pohyb potvrzen
PresunutiPohyb
overDostupnost()
alt
nedostupná
chyba
pohyb zamítnut
ok
ok
zmenLokaci()
ok
pohyb potvrzen
Skladník
:SkladovyPohyb
:Soucastka
:Lokace

## 5.4 Diagram aktivit – Vytvoření pohybu

Diagram zachycuje průběh vytvoření skladového pohybu v systému. Pohyb prochází třemi fázemi řízenými metodami třídy SkladovyPohyb:

- vytvoritPohyb(soucastka, mnozstvi) – uživatel zadá typ pohybu, ID součástky a množství; systém nejprve vyhledá součástku voláním Sklad.najdiSoucastku(id) a získaný objekt předá metodě vytvoritPohyb(), která inicializuje pohyb, přiřadí ID a datum vzniku.
- overitData() – ověří platnost zadaných dat: existenci součástky, dostatečné množství na skladě a platnost lokací u přesunu.
- provestPohyb() – fyzicky provede pohyb podle typu: příjem zavolá zvysMnozstvi(pocet); výdej i přesun nejprve ověří dostupnost voláním Soucastka.overDostupnost() – pokud součástka není dostupná (množství je rezervováno), pohyb je zamítnut – jinak výdej zavolá snizMnozstvi(pocet) a přesun zavolá zmenLokaci(). Pokud data nejsou platná, pohyb je zamítnut.




Obrázek 5.3: Diagram aktivit – Vytvoření pohybu

# 6. Závěr

Projekt se zaměřil na návrh informačního systému pro správu skladu elektronických součástek pomocí UML diagramů. Výsledkem je class diagram s generalizací, kvalifikovanou vazbou, agregací a atributem spojení, dva stavové diagramy a model interakcí zahrnující Use Case, scénáře, sekvenční diagram a diagram aktivit. Jednotlivé modely na sebe navazují a popisují stejný systém z různých pohledů. Projekt pokrývá základní část systému – evidenci součástek, pohybů a projektů – a může být rozšířen o správu objednávek nebo inventurní modul. Umělá inteligence (Claude) pomohla s formátováním a sazbou dokumentu v LaTeXu, urychlila tvorbu PlantUML diagramů a pomohla upřesnit finální podobu dokumentu.

# A. Zdrojové kódy PlantUML diagramů

## A.1 Use Case diagram

```
@startuml 01_use_case
left to right direction
skinparam actorStyle awesome
skinparam nodesep 50
skinparam ranksep 60
skinparam packagePadding 10

actor "Vedoucí skladu" as Vedouci
actor "Skladník" as Skladnik

Vedouci -[hidden]down- Skladnik

rectangle "WMS - Elektronické součástky" {

  package "Projekty" {
    usecase "Spravovat projekt" as UC_PROJEKT
    usecase "Přiřadit součástku\nk projektu" as UC_PRIRADIT
    usecase "Odebrat součástku\nz projektu" as UC_ODEBRAT
  }

  package "Součástky" {
    usecase "Zobrazit zásoby" as UC_ZASOBY
    usecase "Ověřit dostupnost" as UC_DOSTUPNOST
  }

  package "Skladové pohyby" {
    usecase "Vytvořit pohyb" as UC_POHYB
    usecase "Ověřit data pohybu" as UC_OVERIT
    usecase "Příjem na sklad" as UC_PRIJEM
    usecase "Výdej ze skladu" as UC_VYDEJ
    usecase "Přesun součástky" as UC_PRESUN
  }

}

Vedouci -- UC_PROJEKT
Vedouci -- UC_PRIRADIT
Vedouci -- UC_ODEBRAT

Skladnik -- UC_PRIJEM
Skladnik -- UC_VYDEJ
Skladnik -- UC_PRESUN
Skladnik -- UC_ZASOBY
Skladnik -- UC_POHYB

' include
UC_POHYB    ..> UC_OVERIT     : <<include>>
UC_PRIRADIT ..> UC_DOSTUPNOST : <<include>>

' generalizace místo extend - specifické varianty pohybu
UC_PRIJEM --|> UC_POHYB
UC_VYDEJ  --|> UC_POHYB
UC_PRESUN --|> UC_POHYB

@enduml
```

## A.2 Class diagram

```
@startuml
top to bottom direction
skinparam CircledCharacterRadius 0
skinparam CircledCharacterFontSize 0
skinparam defaultFontSize 12
skinparam classFontSize 13
skinparam classFontStyle bold
skinparam ArrowFontSize 14
skinparam nodesep 80
skinparam ranksep 80

abstract class Uzivatel {
  id: int
  jmeno: String
  email: String
}

class Skladnik {
}

class VedouciSkladu {
}

class PrijemPohyb {
  dodavatel: String
  vytvoritPohyb(soucastka, mnozstvi)
}

class VydejPohyb {
  duvod: String
  vytvoritPohyb(soucastka, mnozstvi)
}

class PresunutiPohyb {
  zdrojLokaceId: int
  cilLokaceId: int
  vytvoritPohyb(soucastka, mnozstvi)
  zmenLokaci()
}

abstract class SkladovyPohyb {
  id: int
  datum: Date
  stav: String
  mnozstvi: int
  {abstract} vytvoritPohyb(soucastka, mnozstvi)
  overitData()
  provestPohyb()
}

class Projekt {
  id: int
  nazev: String
  priradSoucastku(Soucastka)
  odeberSoucastku(Soucastka)
}

class ProjektSoucastka {
  mnozstvi: int
}

class Sklad {
  id: int
  nazev: String
  najdiSoucastku(id): Soucastka
}

class Lokace {
  id: int
  oznaceni: String
}

class Soucastka {
  id: int
  nazev: String
  mnozstvi: int
  rezervovaneMnozstvi: int
  stav: String
  zvysMnozstvi(pocet)
  snizMnozstvi(pocet)
  rezervovat(pocet)
  zrusitRezervaci(pocet)
  overDostupnost(): boolean
  odepsat()
}

' -- Generalizace: -down-|> = child ABOVE parent
PrijemPohyb    -down-|> SkladovyPohyb
VydejPohyb     -down-|> SkladovyPohyb
PresunutiPohyb -down-|> SkladovyPohyb
Skladnik       -down-|> Uzivatel
VedouciSkladu  -down-|> Uzivatel

' -- ProjektSoucastka (association class)
Projekt        "1"  --  ProjektSoucastka
ProjektSoucastka    ..> "1" Soucastka

' -- Kvalifikovana vazba
Sklad "[id: int]" -right-> "1" Soucastka : \nnajdiSoucastku

' -- Asociace
VedouciSkladu "1" -right-> "*" Projekt       : \nspravuje
Skladnik      "1" -right-> "*" SkladovyPohyb : \nvytváří
SkladovyPohyb "*"  -->     "1" Sklad         : \nmění stav
SkladovyPohyb "*"  -->     "1" Soucastka     : \nje součástí
Sklad         "1" *--      "*" Lokace        : \nobsahuje
Lokace        "1" o-right- "*" Soucastka     : \nukládá

@enduml
```

## A.3 Stavový diagram – SkladovyPohyb

```
@startuml
top to bottom direction

' Bod 6: míchání stavů podtříd do nadtřídy - pouze jeden stav Proveden
' Bod 3: zakončovací přechody bez názvu události

[*] --> Vytvoren

Vytvoren : entry / vytvoritPohyb(soucastka, mnozstvi)
Vytvoren --> Validovan : Pohyb odeslán

Validovan : entry / overitData()
Validovan : do / kontrolovat data
Validovan : exit / potvrdit výsledek
Validovan --> Proveden  : Pohyb schválen
Validovan --> Zamitnut  : Chyba ověření [neplatná data]
Validovan --> Zamitnut  : Chyba ověření [nedostatečné množství]

Zamitnut : entry / zaznamenat chybu
Zamitnut : do / čekat na notifikaci
Zamitnut : exit / notifikovat uživatele

' Proveden - polymorfně spustí provestPohyb() konkrétní podtřídy
Proveden : entry / provestPohyb()
Proveden : do / overDostupnost() [VydejPohyb | PresunutiPohyb]
Proveden : do / aktualizovat zásoby
Proveden : exit / uložit záznam

' overDostupnost() muze selhat u VydejPohyb a PresunutiPohyb
Proveden --> Zamitnut : součástka nedostupná [overDostupnost() = false]

' zakončovací přechody bez názvu události
Proveden --> [*]
Zamitnut --> [*]

@enduml
```

## A.4 Stavový diagram – Soucastka

```
@startuml
top to bottom direction

[*] --> Dostupna : Přidána do systému / mnozstvi = 0

Dostupna : do / overDostupnost()

' vnitřní přechody - jednorázové akce jako reakce na událost
Dostupna : naskladněno / zvysMnozstvi(pocet)
Dostupna : přiřazena k projektu / rezervovat(pocet)
Dostupna : odebrána z projektu / zrusitRezervaci(pocet)

' výdej: přechod nastane pouze pokud overDostupnost() = true
Dostupna --> Vydana   : výdej [overDostupnost()]
Dostupna --> Odepsana : odepsána [poškozena / zastaralá] / odepsat()

Vydana : entry / snizMnozstvi(pocet)
Vydana --> [*]

Odepsana : entry / snizMnozstvi(pocet)
Odepsana --> [*]

@enduml
```

## A.5 Sekvenční diagram – Vytvoření pohybu

```
@startuml
skinparam defaultFontSize 10
skinparam sequenceArrowThickness 1.5
skinparam sequenceParticipantBorderThickness 1.5
skinparam sequenceGroupBorderThickness 1.5

actor "Skladník" as Uzivatel
participant ":SkladovyPohyb" as Pohyb
note over Pohyb : instance: PrijemPohyb | VydejPohyb | PresunutiPohyb
participant ":Soucastka" as Soucastka
participant ":Lokace" as Lokace

Uzivatel -> Pohyb : vytvoritPohyb(soucastka, mnozstvi)
activate Pohyb

Pohyb -> Pohyb : overitData()

alt [data nejsou platná]
    Pohyb --> Uzivatel : pohyb zamítnut

else [data jsou platná]
    Pohyb -> Pohyb : provestPohyb()

    alt [PrijemPohyb]
        Pohyb -> Soucastka : zvysMnozstvi(pocet)
        activate Soucastka
        Soucastka --> Pohyb : ok
        deactivate Soucastka
        Pohyb --> Uzivatel : pohyb potvrzen

    else [VydejPohyb]
        Pohyb -> Soucastka : overDostupnost()
        activate Soucastka
        alt [součástka nedostupná]
            Soucastka --> Pohyb : chyba
            deactivate Soucastka
            Pohyb --> Uzivatel : pohyb zamítnut
        else [ok]
            Soucastka --> Pohyb : ok
            deactivate Soucastka
            Pohyb -> Soucastka : snizMnozstvi(pocet)
            activate Soucastka
            Soucastka --> Pohyb : ok
            deactivate Soucastka
            Pohyb --> Uzivatel : pohyb potvrzen
        end

    else [PresunutiPohyb]
        Pohyb -> Soucastka : overDostupnost()
        activate Soucastka
        alt [součástka nedostupná]
            Soucastka --> Pohyb : chyba
            deactivate Soucastka
            Pohyb --> Uzivatel : pohyb zamítnut
        else [ok]
            Soucastka --> Pohyb : ok
            deactivate Soucastka
            Pohyb -> Lokace : zmenLokaci()
            activate Lokace
            Lokace --> Pohyb : ok
            deactivate Lokace
            Pohyb --> Uzivatel : pohyb potvrzen
        end
    end
end

deactivate Pohyb

@enduml
```

## A.6 Diagram aktivit – Vytvoření pohybu

```
@startuml
scale 1.5
skinparam defaultFontSize 13
skinparam ArrowThickness 1.5
skinparam ActivityBorderThickness 1.5
skinparam ActivityDiamondBorderThickness 1.5
skinparam ActivityBorderColor #444444
skinparam ActivityDiamondBorderColor #444444
skinparam ArrowColor #444444

start

:Zadat součástku a množství;
:Inicializovat pohyb\n(vytvoritPohyb());
:Ověřit data pohybu\n(overitData());

if (Data platná?) then ([ano])
  :Provést pohyb\n(provestPohyb());

  if (typ == příjem?) then ([ano])
    :Zvýšit množství na skladě\n(Soucastka.zvysMnozstvi(pocet));
  else ([ne])
    if (typ == výdej?) then ([ano])
      :Ověřit dostupnost\n(Soucastka.overDostupnost());
      if (Dostupná?) then ([ano])
        :Snížit množství na skladě\n(Soucastka.snizMnozstvi(pocet));
      else ([ne])
        :Zamítnout pohyb;
      endif
    else ([ne - přesun])
      :Ověřit dostupnost\n(Soucastka.overDostupnost());
      if (Dostupná?) then ([ano])
        :Změnit lokaci součástky\n(PresunutiPohyb.zmenLokaci());
      else ([ne])
        :Zamítnout pohyb;
      endif
    endif
  endif

  :Potvrdit pohyb uživateli;

else ([ne])
  :Zamítnout pohyb;
endif

stop
@enduml
```